CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G01
SYSTEMS INTEGRATION AND ARCHITECTURE 1

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title: GymTrack

Prepared By: Louis Francis Lim

Date of Submission: 06/30/2026

Version: v2.0

Table of Contents

1. Introduction

1.1. Project Title

GymTrack

1.2. Project Background/System Overview

Many small to medium-sized gyms in the Philippines still rely on manual processes such as paper logbooks and handwritten records to manage their members, subscriptions, and attendance. This approach is time-consuming, error-prone, and makes it difficult for gym staff to monitor membership status or generate accurate reports efficiently. Additionally, collecting membership payments manually through cash transactions makes it difficult to track payment history and increases the risk of errors. GymTrack is a web and mobile based Gym Membership Management System that digitalizes and streamlines gym operations.

1.3. Problem Statement

Gym owners and staff face the following challenges with manual membership management

- Difficulty tracking active and expired memberships accurately
- No easy way to monitor daily attendance records
- Manual cash collection with no digital payment tracking or history
- No flexible way for gym owners to create and manage their own subscription tiers
- Delayed or missed notifications for membership renewals
- Security risks from uncontrolled Staff account creation by unauthorized users
- No centralized system accessible on both web and mobile devices

GymTrack addresses these problems by providing a secure, centralized, and accessible membership management solution with a protected Admin registration process, controlled Staff account creation, and integrated online payment support.

1.4. Target Users

User	Role Description
Gym Owner	Full system access. Manages staff accounts, member accounts, subscription plans, payments, and views the dashboard.
Gym Staff	Created by Admin. Manages members, assigns subscription plans, and scans

	QR codes for attendance.
Gym Member	Self-registers. Pays for subscriptions online, views own profile, QR code, attendance history, and payment records.

1.5. Project Objectives/Purpose of the System

General Objective:

To provide gym owners and staff a digital tool for tracking memberships, monitoring daily attendance via QR codes, processing online payments, and managing subscription plans all in one accessible platform.

Specific Objectives:

1. To implement a secure registration system where gym owners register as Admin using a secret registration code, Admins create Staff accounts, and Members self-register freely.

2. To allow gym administrators to create and manage custom subscription tiers with their own pricing and duration.

3. To allow gym staff to manage gym member profiles with unique QR code generation per member.

4. To implement QR code-based attendance check-in and check-out using device cameras.

5. To integrate PayMongo as an online payment gateway supporting GCash, Maya, and credit/debit cards.

6. To send automated in-app notifications for expiring memberships, expired memberships, and successful payments.

7. To provide an Admin dashboard with membership, attendance, and payment summary reports.

8. To deploy the system online and accessible on both web and mobile platforms.

1.6. Scope of the Project

The GymTrack system covers the following:

• Role-based registration (Gym Owner or Member) via a single registration form with a role dropdown

• Staff accounts created by Admin only — not available in the registration dropdown
• Custom subscription plans created, edited, activated, and deactivated by Admin (name, duration in months, price)

• Member self-registration with automatic Member role assignment and unique QR code generation

• Staff and Admin can manage member profiles

• Subscription assignment to members with automatic expiry date computation and status tracking

• QR code attendance tracking scan via Android camera (ZXing) or web browser webcam

• Online payment via PayMongo (GCash, Maya, credit/debit cards)

• PayMongo webhook-based automatic subscription activation upon payment confirmation

• Admin dashboard with key membership, attendance, and payment metrics

• Web application (ReactJS) and mobile application (Android Kotlin)

• Backend REST API built with Spring Boot connected to Supabase

1.7. Limitations of the Project

• Staff accounts cannot be self-registered; they can only be created by an existing Admin.

• The system is limited to PayMongo-supported payment methods (GCash, Maya, Credit/Debit Card) only.

• The system does not support cash payment recording or point-of-sale (POS) integration.

• The system does not include workout planning or fitness tracking features.

• The system does not support biometric or fingerprint-based attendance.

• The mobile application is limited to Android only; iOS is not supported.

• The system is designed for single-branch gym management only.

2. Stakeholders and Overall Description

2.1. Stakeholders and Users

Stakeholder	**Type**	**Role**
Gym Owner	Primary User	Manages entire system: staff, members, plans, payments, dashboard
Gym Staff	Primary User	Manages members, assigns plans, scans QR codes
Gym Member	Primary User	Registers, pays online, views personal records
Developer	Stakeholder	Processes online payments and sends webhook confirmation

2.2. Product Perspective

GymTrack is a multi-platform information management system composed of three integrated components. Both the web and mobile applications connect to the same SpringBoot, ensuring consistent data across platforms. PayMongo is integrated into the backend to handle secure online payment processing and webhook confirmation.

2.3. Operating Environment

Backend:

• Java

• Spring Boot

• Supabase

Web Application:

• ReactJS

• Compatible with modern browsers (Chrome, Firefox, Edge)

Mobile Application:

• Android 8.0 or higher

• Android Studio for development

• Kotlin programming language

• Retrofit for API communication

• ZXing library for camera-based QR code scanning

• Camera permission required for QR code scanning

2.4. Assumptions, Dependencies, and Constraints

Assumptions:

• The Gym Owner who registers is the legitimate owner of the gym

• Only Admin can create Staff accounts

• Members self-register and automatically receive the Member role

• The gym has a stable internet connection

• Members have access to GCash, Maya, or a credit/debit card for payment

• Staff devices have functional cameras for QR code scanning

Dependencies:

• Spring Boot and Supabase for backend services

• PayMongo API for online payment processing

• ReactJS ecosystem

• Android Kotlin with Retrofit and ZXing library

Constraints:

• Staff accounts are not available in the registration dropdown

• Only PayMongo payments are supported no cash recording or POS integration

• The mobile app supports Android only no iOS version

• The system does not include workout planning or fitness tracking features

- The system does not support biometric or fingerprint-based attendance
- The system is designed for single-branch gym management only

3. System Features and Functional Requirements

Describe each major feature of the system and its functional requirements.

3.1. Feature 1: User Authentication and Account Management

ID	Requirement
Fr-001	The system shall provide a registration form with a role dropdown: “Gym Owner” or “Member” only.
Fr-002	The system shall create an Admin account for “Gym Owner” selection and a Member account for “Member” selection.
Fr-003	The system shall NOT include “Staff” in the registration dropdown.
Fr-004	The system shall allow only Admin to create, edit, and deactivate Staff accounts.
Fr-005	The system shall authenticate users via email and password.
Fr-006	The system shall restrict features based on user role (Admin, Staff, Member).
Fr-007	The system shall allow Admin to deactivate Staff or Member accounts.
Fr-008	The system shall allow all users to log out.

3.2. Feature 2: Member Management

ID	Requirement
FR-009	The system shall auto-create a member profile with a unique QR code upon registration.
FR-010	The system shall allow Staff and Admin to view, search, filter, and edit members.
FR-011	The system shall display a member's subscription status and payment history on their profile.
FR-012	The system shall allow Members to view their own profile, QR code, and attendance history.

3.3. Feature 3: Subscription Plan Management

ID	Requirement
FR-013	The system shall allow only Admin to create and edit subscription plans (name, duration in months, price).
FR-014	The system shall allow Admin to activate or deactivate plans.
FR-015	The system shall display only active plans for assignment or payment.
FR-016	The system shall allow Staff and Admin to assign a plan to a member with an auto-calculated expiry date.
FR-017	The system shall display membership status as “Active,” “Expiring Soon,” or “Expired.”

3.4. Feature 4: Online Payment

ID	Requirement
FR-018	The system shall allow Members to select a plan and pay online via PayMongo (GCash, Maya, cards).
FR-019	The system shall redirect Members to PayMongo and receive webhook confirmation after payment.
FR-020	The system shall verify webhook signatures and activate the subscription.
FR-021	The system shall store payment records (amount, method, date, status, reference) immutably.
FR-022	The system shall allow Admin to view all payment transactions.

3.5. Feature 5: Attendance Tracking

ID	Requirement
FR-023	The system shall allow Members to select a plan and pay online via PayMongo (GCash, Maya, cards).
FR-024	The system shall redirect Members to PayMongo and receive webhook confirmation after payment.
FR-025	The system shall allow Staff to scan QR codes using a webcam on the ReactJS web application.
FR-026	The system shall record check-in date and time on a successful scan.
FR-027	The system shall allow Staff to manually record a Member's check-out time.
FR-028	The system shall display member name, plan, and status after scanning.
FR-029	The system shall deny check-in if the subscription is expired or unpaid.
FR-030	The system shall allow filtering attendance logs by date or member.

3.6. Feature 6: Admin Dashboard

ID	Requirement
FR-031	The system shall display total members, active subscriptions, and expired memberships.
FR-032	The system shall display today's check-in count and total payments collected.
FR-033	The system shall allow Admin to filter attendance logs and manage Staff/Member accounts.

4. Non-Functional Requirements

4.1 Performance

ID	Requirement
NFR-001	Pages and API responses shall load within 3 seconds under normal conditions.
NFR-002	The system shall support at least 20 concurrent users.
NFR-003	Subscription activation shall complete within 5 seconds of webhook receipt.

4.2 Security

ID	Requirement
NFR-004	The system shall hash all user passwords before storing them in the database.
NFR-005	The system shall secure all API endpoints using JWT authentication.

NFR-006	The system shall restrict Staff account creation to Admin users only.
NFR-007	The system shall verify PayMongo webhook signatures before processing payment events.
NFR-008	The system shall enforce role-based access control.

4.3 Usability

ID	Requirement
NFR-009	The system shall provide a simple registration form with a clear role dropdown.
NFR-010	The system shall display clear and descriptive error messages for invalid inputs.
NFR-011	The system shall show clear payment status indicators at each step of the PayMongo checkout flow.
NFR-012	The system shall be intuitive and easy to navigate on both web and mobile interfaces.

4.4 Compatibility

ID	Requirement
NFR-013	The web application shall function correctly on Chrome, Firefox, and Edge browsers.
NFR-014	The mobile application shall support Android 8.0 (Oreo) and above.
NFR-015	Both the web and mobile applications shall use the same Spring Boot REST API.

4.5 Reliability

ID	Requirement
NFR-016	The system shall maintain at least 95% uptime.
NFR-0147	The system shall store all data consistently and without loss during normal operations.
NFR-018	The system shall not allow payment records to be edited or deleted after confirmation.

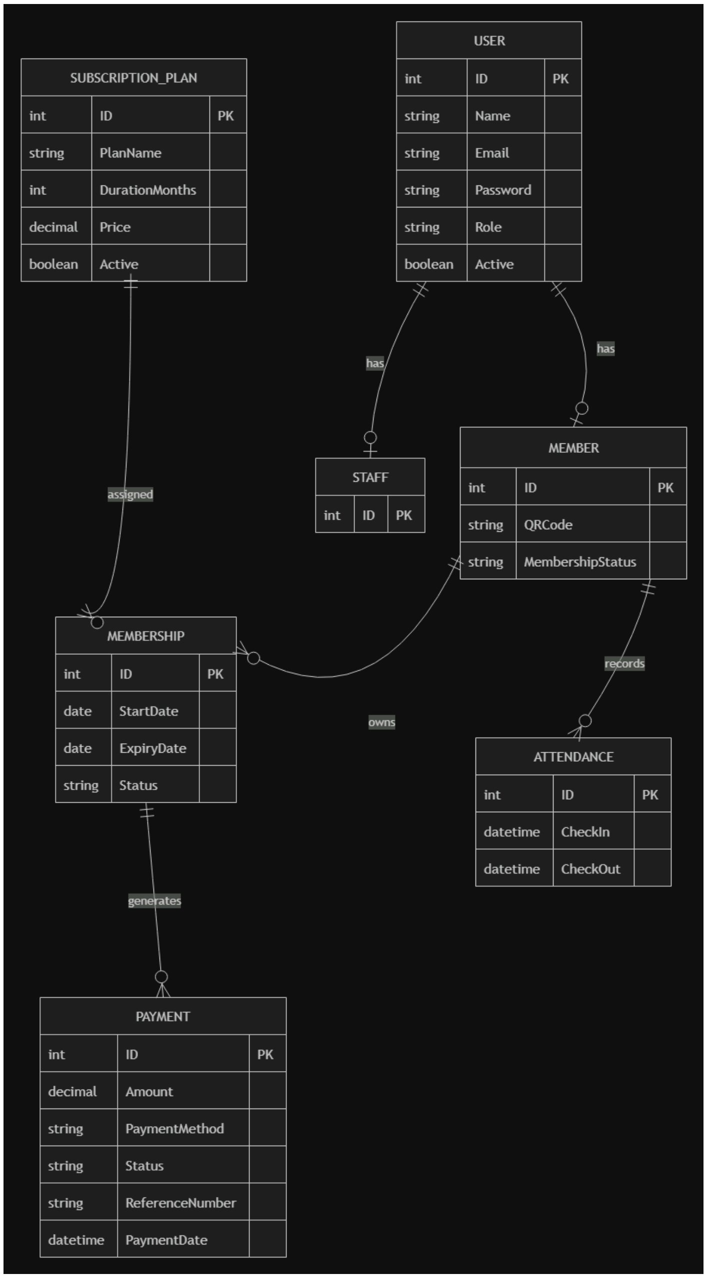
4.6 Maintainability

ID	Requirement
NFR-019	The backend shall follow a Controller-Service-Repository architecture pattern.
NFR-020	The API shall follow RESTful conventions for endpoint naming and HTTP methods.
NFR-021	PayMongo integration logic shall be isolated in a dedicated service class.
NFR-022	Role-based access control shall be managed through Spring Security configuration.

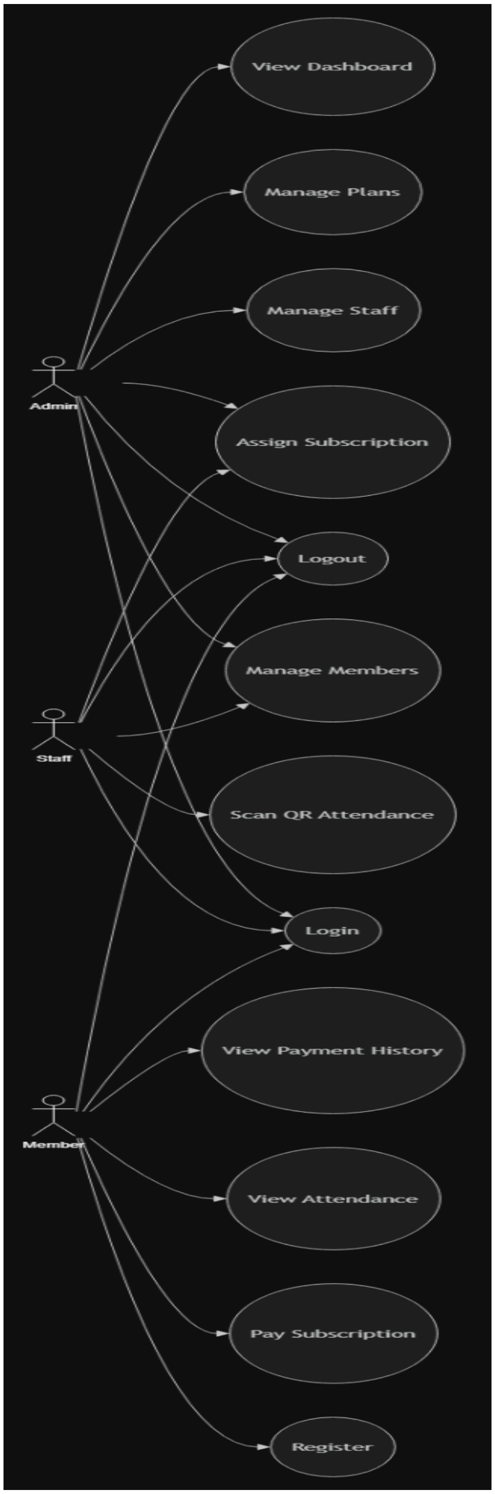
5. System Models (Diagrams)

Insert the necessary diagrams for the system:

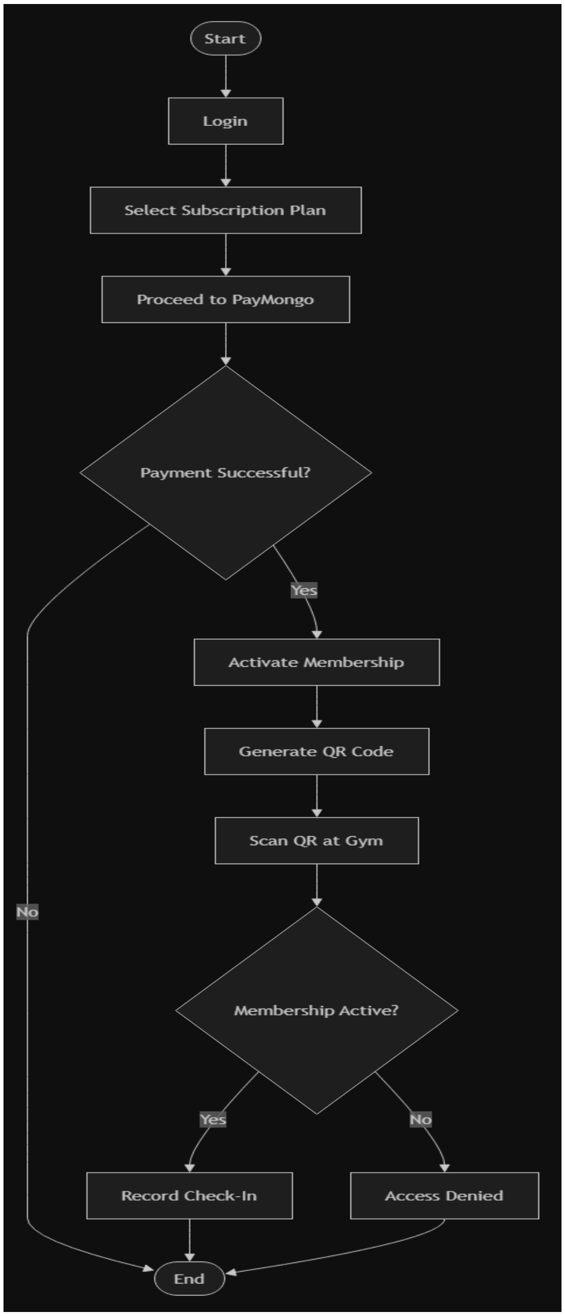
5.1. Entity +Relationship Diagrams



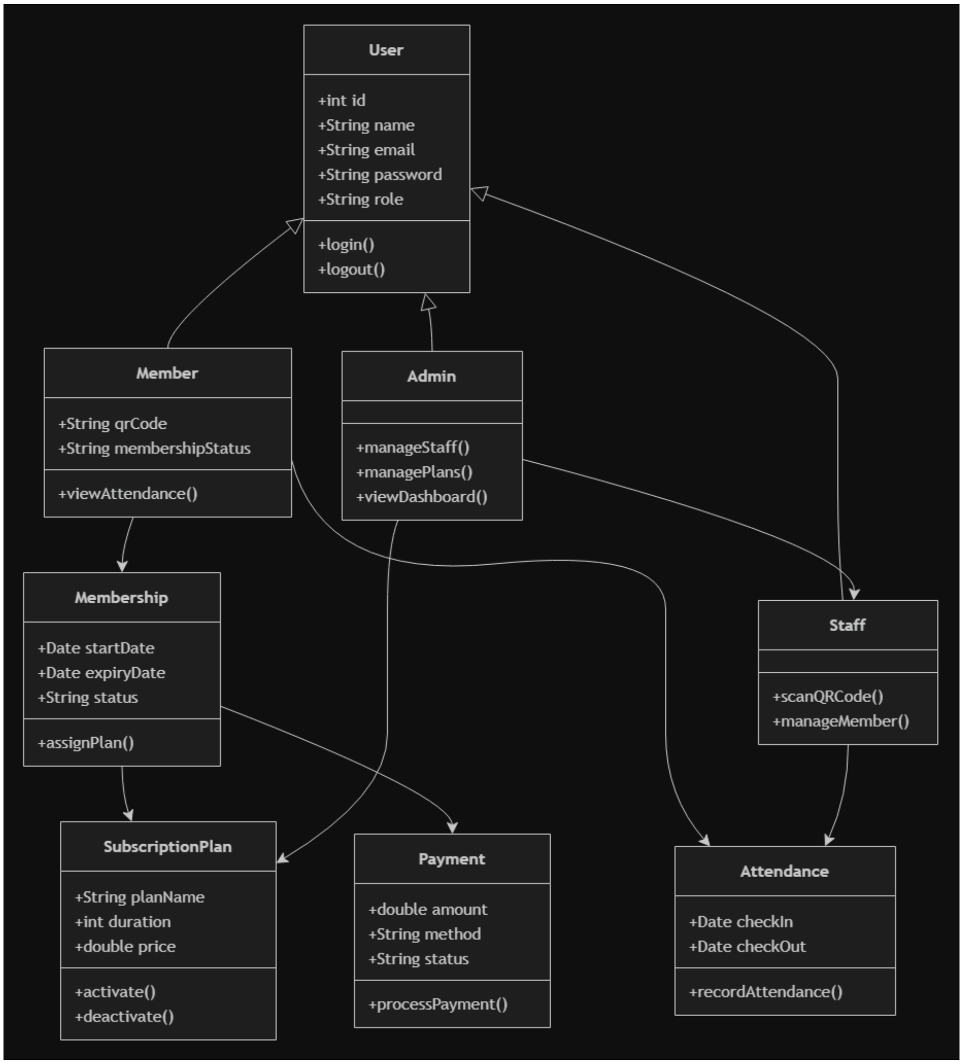
5.2. Use Case Models



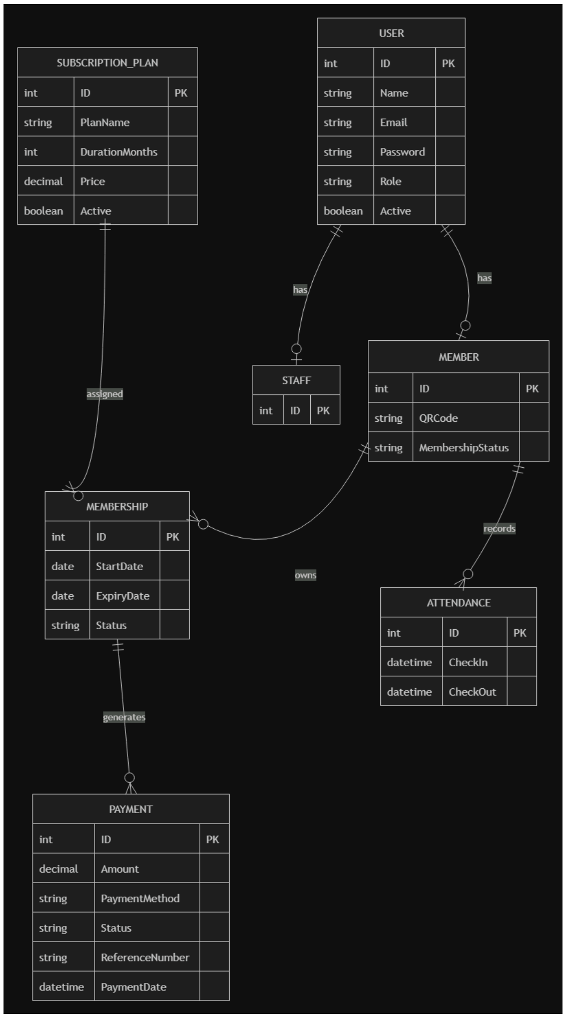
5.3. Activity Diagram



5.4. Class Diagram



5.5. Sequence Diagram



6. Appendices

Include any additional information, references, or support materials.

7. Revision History Table

Version	Date	Changes
v1.0	6/23/26	Initial SRS submitted
v2.0	6/30/26	Major revision of the SRS. Reorganized the document structure, expanded the functional and non-functional requirements, added detailed feature descriptions, introduced stakeholders and target users, updated the system architecture to use Spring Boot with Supabase, refined the project scope and assumptions, and added the required system model sections (Entity Relationship Diagram, Use Case Diagram, Activity Diagram, Class Diagram, and Sequence Diagram).

8. Initial Project Timeline

Week	Planned Activity
Week 1	Proposal preparation and approval
Week 2	Requirements refinement
Week 3	UI and database design
Week 4	Backend setup

9. References

Auth0. (n.d.). JWT introduction. https://jwt.io/introduction

BCrypt. (n.d.). BCrypt password hashing function. https://github.com/jeremyh/jBCrypt
Google. (n.d.). Android Developers. https://developer.android.com/

JetBrains. (n.d.). Kotlin documentation. https://kotlinlang.org/docs/home.html Oracle. (n.d.). Java documentation. https://docs.oracle.com/en/java/

PayMongo. (n.d.). PayMongo API documentation. https://developers.paymongo.com/

React. (n.d.). React documentation. https://react.dev/

Spring. (n.d.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/documentation.html

Supabase. (n.d.). Supabase documentation. https://supabase.com/docs

ZXing. (n.d.). ZXing ("Zebra Crossing") documentation. https://github.com/zxing/zxing
Mozilla. (n.d.). MDN Web Docs. https://developer.mozilla.org/